

SAPTAMANA 5

PRINT-int : creare fisiere: print-int.l , print-int.y, print-int.in (fisier cu date de intrare)si Makefile (se executa cu: make) in acelasi folder

```
/* print-int.l */
%%
[0-9]+ {sscanf (yytext, "%d", &yyval);
        return (INTEGER);
      }
\n      return (NEWLINE);
.       return (yytext[0]);
%%
int yywrap() { return 1; }

-----

/* print-int.y */
%token INTEGER NEWLINE
%%
lines: /* empty */
      | lines NEWLINE
      | lines line NEWLINE {printf("=%d\n", $2);}
      | error NEWLINE {yyerror("Reenter:"); yyerrok;}
      ;
line : INTEGER {$$ = $1;}
      ;
%%
#include "lex.yy.c"
main() {
    return yyparse();
}
int yyerror( char *s ) { fprintf( stderr, "%s\n", s); }
```

print-int.in

```
45
66
111111111111
```

FISIERUL Makefile

```
print-int: y.tab.c lex.yy.c
        gcc -o print-int y.tab.c
y.tab.c: print-int.y lex.yy.c
        yacc print-int.y
lex.yy.c: print-int.l
        lex print-int.l
clean:
        rm -f lex.yy.c y.tab.c print-int
```

Executie:

\$ make

\$./print-int <print-int.in >print-int.out

Expression Evaluation YACC S6

```
%{
#include<stdio.h>
%}
%token DIGIT
%%
S      : E '\n' {printf("%d\n",$1);return 1;}
      ;
E      : E '+'    T      { $$ = $1 + $3 ; }
      | T
      ;
T      : T '*'    F      { $$ = $1 * $3; }
      | F
      ;
F      : '(' E    ')'    { $$=$2; }
      | DIGIT
      ;
%%

yylex(){
int c;
c=getchar();
if(isdigit(c))
{
    yylval=c-'0';
    return DIGIT;
}

return c;
}

main()
{
printf("Enter the exp:");
yyparse();
return 0;
}
```

EXECUTIE:

```
students@ccflab-desktop:~$ yacc aa.y
students@ccflab-desktop:~$ gcc y.tab.c -ly
students@ccflab-desktop:~$ ./a.out
Enter the exp:2+5*5
```

Expression Evaluation - Yacc Program - Compiler Design

Program:

```
// Yacc Program: expr.y
%{
#include"stdio.h"
%}
%token DIGIT
%%
S      : E '\n' {printf("%d\n",$1);return 1;}
      ;
E      : E '+' T { $$ = $1 + $3 ;}
      | T
      ;
T      : T '-' FF { $$ = $1 - $3 ;}
      | FF
      ;
FF     : FF '/' FFF { $$ = $1 / $3 ;}
      | FFF;
FFF    : FFF '*' F { $$ =$1*$3 ;}
      | F
      ;
F      : '(' E ')' { $$ = $2 ;}
      | DIGIT
      ;

%%
yylex()
{
int c;
c=getchar();
if(isdigit(c))
{
    yylval=c-'0';
    return DIGIT;
}
return c;
}
main()
{ printf("Enter the exp:"); yyparse();
}
```

EXECUTIE:

```
nn@linuxmint ~ $ lex io.l
nn@linuxmint ~ $ yacc expr.y
nn@linuxmint ~ $ gcc y.tab.c -ll -ly
nn@linuxmint ~ $ ./a.out
Enter the exp:5*(5+1)-9/2
```

```

| ID
| NUM
;
E2 : E'<'E
| E'>'E
| E LE E
| E GE E
| E EQ E
| E NE E
| E OR E
| E AND E
| ID
| NUM
;
%%

```

```

#include "lex.yy.c"

```

```

main()
{
printf("Enter the statement: ");
yyparse();
}

```

EXECUTIE:

```

nn@linuxmint ~ $ lex if.l
nn@linuxmint ~ $ yacc if.y
nn@linuxmint ~ $ gcc y.tab.c -ll -ly
nn@linuxmint ~ $ ./a.out
Enter the statement: if(i>) then i=1;
syntax error
nn@linuxmint ~ $ ./a.out
Enter the statement: if(i>8) then i=1;
Input accepted.
nn@linuxmint ~ $

```

Parser for IF-THEN-ELSE Statements - YACC Program - Compiler Design

Program:

```

(Lex Program: ift.l)
alpha [A-Za-z]
digit [0-9]

```

```

%%
[ \t\n]
if      return IF;
then    return THEN;
else    return ELSE;
{digit}+    return NUM;
{alpha}({alpha}|{digit})*    return ID;
"<="      return LE;
">="      return GE;
"=="      return EQ;
"!="      return NE;
"||"      return OR;
"&&"      return AND;
.         return yytext[0];
%%

```

(Yacc Program: **ift.y**)

```

%{
#include <stdio.h>
#include <stdlib.h>
%}

%token ID NUM IF THEN LE GE EQ NE OR AND ELSE
%right '='
%left AND OR
%left '<' '>' LE GE EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS
%left '!'

%%

S      : ST {printf("Input accepted.\n");exit(0);};
ST     : IF '(' E2 ')' THEN ST1 ';' ELSE ST1 ';'
        | IF '(' E2 ')' THEN ST1 ';'
        ;
ST1    : ST
        | E
        ;
E      : ID '=' E
        | E '+' E
        | E '-' E
        | E '*' E
        | E '/' E

```

```

| E'<'E
| E'>'E
| E LE E
| E GE E
| E EQ E
| E NE E
| E OR E
| E AND E
| ID
| NUM
;

```

```

E2 : E'<'E
| E'>'E
| E LE E
| E GE E
| E EQ E
| E NE E
| E OR E
| E AND E
| ID
| NUM
;

```

```

%%
#include "lex.yy.c"
main()
{
    printf("Enter the exp: ");
    yyparse();
}

```

EXECUTIE:

```

students@cselab-desktop:~$ lex ift.lex
students@cselab-desktop:~$ yacc ift.y
students@cselab-desktop:~$ gcc y.tab.c -ll -ly
students@cselab-desktop:~$ ./a.out
Enter the exp: if(a==1) then b=1; else b=2;
Input accepted.
students@cselab-desktop:~$

```

SAPTAMANA 6

PARSER

Parser for FOR Loop Statements - YACC Program - Compiler Design Program:

// Lex file: for.l

alpha [A-Za-z]

digit [0-9]

%%

[\t \n]

for return FOR;

{digit}+ return NUM;

{alpha}({alpha}|{digit})* return ID;

"<=" return LE;

">=" return GE;

"==" return EQ;

"!=" return NE;

"||" return OR;

"&&" return AND;

return yytext[0];

%%

// Yacc file: for.y

{

#include <stdio.h>

#include <stdlib.h>

}

%token ID NUM FOR LE GE EQ NE OR AND

%right "="

%left OR AND

%left '>' '<' LE GE EQ NE

%left '+' '-'

%left '*' '/'

%right UMINUS

%left '!'

%%

```
S      : ST {printf("Input accepted\n"); exit(0);}
ST     : FOR '(' E ';' E2 ';' E ')' DEF
      ;
DEF    : '{' BODY '}'
      | E ';'
      | ST
      |
      ;
BODY   : BODY BODY
      | E ';'
      | ST
      |
      ;

E      : ID '=' E
      | E '+' E
      | E '-' E
      | E '*' E
      | E '/' E
      | E '<' E
      | E '>' E
      | E LE E
      | E GE E
      | E EQ E
      | E NE E
      | E OR E
      | E AND E
      | E '+' '+'
      | E '-' '-'
      | ID
      | NUM
      ;

E2     : E '<' E
      | E '>' E
      | E LE E
      | E GE E
      | E EQ E
      | E NE E
```

```

        | E OR E
        | E AND E
    ;

%%

#include "lex.yy.c"
main() {
    printf("Enter the expression:\n");
    yyparse();
}

EXECUTIE:
nn@linuxmint ~ $ lex for.l
nn@linuxmint ~ $ yacc for.y
conflicts: 25 shift/reduce, 4 reduce/reduce
nn@linuxmint ~ $ gcc y.tab.c -ll -ly
nn@linuxmint ~ $ ./a.out
Enter the expression:
for(i=0;i<n;i++)
i=i+1;
Input accepted
nn@linuxmint ~ $

```

Parser for WHILE Statements - YACC Program - Compiler Design

Program:

```

(wh.l)

alpha [A-Za-z]
digit [0-9]
%%
[ \t\n]
while      return WHILE;
{digit}+   return NUM;
{alpha}({alpha}|{digit})*   return ID;
"<="      return LE;
">="      return GE;
"=="      return EQ;
"!="      return NE;
"||"      return OR;

```

```

"&&"    return AND;
.        return yytext[0];
%%

```

```

(wh.y)

```

```

%{
#include <stdio.h>
#include <stdlib.h>
}%
%token ID NUM WHILE LE GE EQ NE OR AND
%right '='
%left AND OR
%left '<' '>' LE GE EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS
%left '!'
%%
S      : ST1 {printf("Input accepted.\n");exit(0);};
ST1    : WHILE '(' E2 ')' '{' ST '}'
ST      : ST ST
        | E ';'
        ;
E       : ID '=' E
        | E '+' E
        | E '-' E
        | E '*' E
        | E '/' E
        | E '<' E
        | E '>' E
        | E LE E
        | E GE E
        | E EQ E
        | E NE E
        | E OR E
        | E AND E
        | ID
        | NUM
        ;
E2      : E '<' E
        | E '>' E

```

```

        | E LE E
        | E GE E
        | E EQ E
        | E NE E
        | E OR E
        | E AND E
        | ID
        | NUM
    ;

```

```

%%
#include "lex.yy.c"
main()
{
    printf("Enter the exp: ");
    yyparse();
}

```

EXECUTIE:

```

students@ccflab-desktop:~$ lex wh.l
students@ccflab-desktop:~$ yacc wh.y
conflicts: 2 shift/reduce
students@ccflab-desktop:~$ gcc y.tab.c -ll -ly
students@ccflab-desktop:~$ ./a.out
Enter the exp: while(a>1){ b=1;}
Input accepted.

```

Parser for DO WHILE Statements - YACC Program - Compiler Design

Program:

```

(dowh.l)
alpha [A-Za-z]
digit [0-9]
%%
[ \t\n]
do      return DO;
while   return WHILE;
{digit}+    return NUM;
{alpha}({alpha}|{digit})*    return ID;
"<="      return LE;
">="      return GE;
"=="      return EQ;
"!="      return NE;
"||"      return OR;

```

```

"&&"    return AND;
.        return yytext[0];
%%

```

(dowh.y)

```

%{
#include <stdio.h>
#include <stdlib.h>
%}
%token ID NUM DO WHILE LE GE EQ NE OR AND
%right '='
%left AND OR
%left '<' '>' LE GE EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS
%left '!'

```

```

%%

```

```

S : ST1 {printf("Input accepted.\n");exit(0);};

```

```

ST1 : DO '{' ST '}' WHILE '(' E2 ')' ';' ;

```

```

ST : ST ST
    | E ';'
    ;

```

```

E : ID '=' E
    | E '+' E
    | E '-' E
    | E '*' E
    | E '/' E
    | E '<' E
    | E '>' E
    | E LE E
    | E GE E
    | E EQ E
    | E NE E
    | E OR E
    | E AND E
    | ID
    | NUM
    ;

```

```

E2 : E '<' E
    | E '>' E

```

```

| E LE E
| E GE E
| E EQ E
| E NE E
| E OR E
| E AND E
| ID
| NUM
;

%%

```

```

#include "lex.yy.c"
main()
{
    printf("Enter the exp: ");
    yyparse();
}

```

EXECUTIE

```

students@ccflab-desktop:~$ lex dowh.l
students@ccflab-desktop:~$ yacc dowh.y
conflicts: 2 shift/reduce
students@ccflab-desktop:~$ gcc y.tab.c -ll -ly
students@ccflab-desktop:~$ ./a.out
Enter the exp: do{a=b+1;}while(a<10);
Input accepted.
students@ccflab-desktop:~$

```

Parser for SWITCH Statements - YACC Program - Compiler Design

Program:

```

(sw.l)
alpha [A-Za-z]
digit [0-9]
%%
[ \t\n]
switch      return SWITCH;
case return CASE;
break return BREAK;
default return DEFAULT;

```

```

{digit}+    return NUM;
{alpha}({alpha}|{digit})*    return ID;
"<="    return LE;
">="    return GE;
"=="    return EQ;
"!="    return NE;
"||"    return OR;
"&&"    return AND;
.    return yytext[0];
%%

```

```

(sw.y)
%{
#include <stdio.h>
#include <stdlib.h>
%}
%token ID NUM SWITCH CASE DEFAULT BREAK LE GE EQ NE OR AND ELSE
%right '='
%left AND OR
%left '<' '>' LE GE EQ NE
%left '+' '-'
%left '*' '/'
%right UMINUS
%left '!'
%%

```

```

S      : ST {printf("Input accepted.\n");exit(0);}
      ;

```

```

ST     : SWITCH '(' ID ')' '{' B '}'
      ;

```

```

B      : C
      | C D
      ;

```

```

C      : C C
      | CASE NUM ':' E ';'
      | BREAK ';'
      ;

```

```

D      : DEFAULT ':' E ';' BREAK ';'
      ;

```



```

E      : ID='E
      | E+'E
      | E-'E
      | E'*'E
      | E'/'E
      | E'<'E
      | E'>'E
      | E LE E
      | E GE E
      | E EQ E
      | E NE E
      | E OR E
      | E AND E
      | ID
      | NUM
      ;

```

```
%%
```

```
#include "lex.yy.c"
```

```

main()
{
printf("Enter the exp: ");
yyparse();
}

```

EXECUTIE:

```

students@cselab-desktop:~$ lex sw.l
students@cselab-desktop:~$ yacc sw.y
conflicts: 2 shift/reduce
students@cselab-desktop:~$ gcc y.tab.c -ll -ly
students@cselab-desktop:~$ ./a.out
Enter the exp: switch(a)
{
case 1: a=1;
      break;
case 2: a=2;
      break;
default:a=0;
break;
}

```

SAPTAMANA 7

```
}  
Input accepted.  
students@cselab-desktop:~$
```

Parser for Function Definition - YACC Program - Compiler Lab

Program :

(Lex Program : fundf.l)

```
alpha [A-Za-z]  
digit [0-9]  
  
%%  
  
[\\t \\n]          ;  
int|float|void|char      return TYPE;  
return                return RETURN;  
{digit}+             return NUM;  
{alpha}({alpha}|{digit})*  return ID;  
.  
                        return yytext[0];  
  
%%
```

(Yacc Program : fundf.y)

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
%token TYPE RETURN ID NUM  
%right "="  
%left '+' '-'  
%left '*' '/'  
%right UMINUS  
%left '!'  
  
%%  
  
S      : FUN {printf("Input accepted\\n"); exit(0);}  
FUN    : TYPE ID '(' PARAM ')' '{' BODY '}'  
      ;
```

```

PARAM      : PARAM ',' TYPE ID
    | TYPE ID
    |
    ;
BODY       : BODY BODY
    | PARAM ';'
    | E ';'
        | RETURN E ';'
        |
    ;
E         : ID '=' E
    | E '+' E
    | E '-' E
    | E '*' E
    | E '/' E
    | ID
    | NUM
    ;

```

```
%%
```

```
#include "lex.yy.c"
```

```
main()
```

```

{
    printf("Enter the expression:\n");
    yyparse();
}

```

```
EXECUTIE_:
```

```
nn@linuxmint ~ $ lex fund.l
```

```
nn@linuxmint ~ $ yacc fund.y
```

```
conflicts: 17 shift/reduce, 11 reduce/reduce
```

```
nn@linuxmint ~ $ gcc y.tab.c -ll -ly
```

```
nn@linuxmint ~ $ ./a.out
```

```
Enter the expression:
```

```
float sum(int term)
```

```

{
    float result;
    result=result+term;
    return result;
}

```

```
}  
Input accepted  
nn@linuxmint ~ $
```

Parser for SQL Query Select - YACC Program - Compiler Design

Program:

```
// Lex Program :sq.l
```

```
alpha [A-Za-z]
```

```
digit [0-9]
```

```
%%
```

```
[ \t\n]
```

```
select          return SELECT;
```

```
distinct        return DISTINCT;
```

```
from            return FROM;
```

```
where           return WHERE;
```

```
like            return LIKE;
```

```
desc           return DESC;
```

```
asc            return ASC;
```

```
"group by"      return GROUP;
```

```
having          return HAVING;
```

```
"order by"      return ORDER;
```

```
or              return OR;
```

```
and             return AND;
```

```
{digit}+        return NUM;
```

```
{alpha}({alpha}|{digit})* return ID;
```

```
"<="           return LE;
```

```
">="           return GE;
```

```
"=="           return EQ;
```

```
"!="           return NE;
```

```
.              return yytext[0];
```

```
%%
```

```
// Yacc Program: sq.y
```

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
%}
```

```
%token ID NUM SELECT DISTINCT FROM WHERE LE GE EQ NE OR AND LIKE GROUP
HAVING ORDER ASC DESC
%right '='
%left AND OR
%left '<' '>' LE GE EQ NE
```

```
%%
```

```

    S          : ST1';' {printf("INPUT ACCEPTED....
\n\t\t...www.2k8cse.cu.cc\n");exit(0);};
    ST1        : SELECT attributeList FROM tableList ST2
                | SELECT DISTINCT attributeList FROM tableList ST2
                ;
    ST2        : WHERE COND ST3
                | ST3
                ;
    ST3        : GROUP attributeList ST4
                | ST4
                ;
    ST4        : HAVING COND ST5
                | ST5
                ;
    ST5        : ORDER attributeList ST6
                |
                ;
    ST6        : DESC
                | ASC
                |
                ;
    attributeList :      ID','attributeList
                | '*'
                | ID
                ;
    tableList    : ID',' tableList
                | ID
                ;
    COND        : COND OR COND
                | COND AND COND
                | E
                ;
    E           : F '=' F
                | F '<' F

```

```

        | F '>' F
        | F LE F
        | F GE F
        | F EQ F
        | F NE F
        | F OR F
        | F AND F
        | F LIKE F
    ;

F      : ID
        | NUM
    ;

```

```
%%
```

```
#include "lex.yy.c"
```

```
#include <ctype.h>
```

```
main()
```

```
{
    printf("Enter the query:");
    yyparse();
}
```

```
EXECUTIE:
```

```
nn@linuxmint ~ $ lex sq.l
```

```
nn@linuxmint ~ $ yacc sq.y
```

```
nn@linuxmint ~ $ gcc y.tab.c -ll -ly
```

```
nn@linuxmint ~ $ ./a.out
```

```
Enter the query:select name,address from emp where age>20 group by name
having age<40 order by name desc;
```

```
INPUT ACCEPTED...
```

```
...www.2k8cse.cu.cc
```

```
nn@linuxmint ~ $
```

~~Intermediate Code Generator for Arithmetic Expression - Yacc Program - Compiler Lab~~

~~Program:~~

~~(Lex Program : intar.l)~~

~~ALPHA [A-Za-z]~~

~~DIGIT [0-9]~~

Calculator mai avansat: handles parentheses and order of operations (copied from the O'Reilly book). [ch302.y](#) and [ch302.1](#),

[ch302.y](#) and

```
%{
#include <stdio.h>
%}

%token NAME NUMBER
%left '-' '+'
%left '*' '/'
%nonassoc UMINUS

%%
statement:      NAME '=' expression          { printf("pretending to assign %s
the value %d\n", $1, $3); }
              ;
              |      expression              { printf("= %d\n", $1); }
              ;
expression:      expression '+' expression    { $$ = $1 + $3;
expression.\n");                             printf ("Recognized '+'
              |      expression '-' expression { $$ = $1 - $3;
expression.\n");                             printf ("Recognized '-'
              |      expression '*' expression { $$ = $1 * $3;
expression.\n");                             printf ("Recognized '*'
              |      expression '/' expression { if ($3 == 0)
                                                yyerror ("divide by zero");
                                                else
                                                $$ = $1 / $3;
                                                printf ("Recognized '/'
expression.\n");
              |      '-' expression %prec UMINUS { $$ = - $2;
expression.\n");                             printf ("Recognized negation.\n");
              |      '(' expression ')'          { $$ = $2;
expression.\n");                             printf ("Recognized parenthesized
              |      NUMBER                      { $$ = $1;
                                                printf ("Recognized a number.\n");
              ;
                                                }
%%
int main (void) {
    return yyparse();
}

/* Added because panther doesn't have liby.a installed. */
```



```
int yyerror (char *msg) {
    return fprintf (stderr, "YACC: %s\n", msg);
}
```

ch302.1

```
%{
#include "ch302.tab.h"
extern int yylval;
}%

%%
[0-9]+ { yylval = atoi (yytext);
        printf ("scanned the number %d\n", yylval);
        return NUMBER; }
[ \t]  { printf ("skipped whitespace\n"); }
\n     { printf ("reached end of line\n");
        return 0;
        }
.      { printf ("found other data \"%s\"\n", yytext);
        return yytext[0];
        }

%%
```

Attempt at reading English sentences: [words2.y](#) and [words2.1](#)

words2.y

```
%{
#include <stdio.h>

int at_end = 0;
extern int yychar;
}%

%token ARTICLE VERB NOUN ADJECTIVE ADVERB PREPOSITION END
%start sentence

%%
sentence:      nphrase VERB termunct          { printf ("Simple noun-verb
sentence.\n");
                                                    YYACCEPT;}
      |      nphrase VERB vmodifier termunct    { printf ("Sentence with
modified verb\n");
                                                    YYACCEPT;}
      |      nphrase VERB nphrase termunct    { printf ("Sentence with
object\n");
                                                    YYACCEPT;}
      |      END
                                                    { printf ("Got EOF from lex.\n");
                                                    YYACCEPT;}

/* All these YYACCEPTS are needed so yyparse will return
immediately,
rather than waiting for the first token of the next
```

```

sentence.  They
                                wouldn't be necessary if the main program were only calling
yyvsparse() once.  */
;
termpunct:    '.'                ;
              '|'                ;
;
nphrase:      modifiednoun       ;
              |                   { printf ("\tGot an article\n"); }
              ARTICLE modifiednoun
;
modifiednoun: NOUN
              |                   { printf ("\tmodified noun\n"); }
              nmodifier modifiednoun
;
nmodifier:    ADJECTIVE          ;
              |                   { printf ("\tadded an adverb to a noun
modifier\n"); }
              ADVERB nmodifier
;
vmodifier:    ADVERB             ;
              |                   { printf ("\tadded an adverb to a verb
modifier\n"); }
              ADVERB vmodifier
              |                   { printf ("\tprepositional phrase\n"); }
              PREPOSITION nphrase
;

%%
int main (void) {
    while (! at_end) {
       yyvsparse();
    }
    printf ("Wasn't that fun?\n");
}
/* Added because panther doesn't have liby.a installed. */
int yyerror (char *msg) {
    return fprintf (stderr, "YACC: %s; yychar=%d\n", msg, yychar);
}

```

words2.1

```

%{
#include "words2.tab.h"
extern int yylval;
extern int at_end;
}%

%%

<<EOF>>      {
               at_end = 1;
               return END;
               }

" "          |
\n           |
"\t"         ;

[Tt]he      |

```

```

[Aa]n?          return ARTICLE;

go(es)?         |
jumps?          |
runs?           |
likes?          |
eats?           return VERB;

dogs?           |
cats?           |
fish            |
fox(es)?        |
moose           return NOUN;

quick           |
slow            |
lazy            |
clever          |
smart           |
stupid          |
brown           |
black           |
blue            |
red             |
orange          |
white           |
big             |
small           return ADJECTIVE;

quickly         |
easily          |
openly          |
slowly          return ADVERB;

over            |
under           |
around          |
through         |
between         return PREPOSITION;

.               return yytext[0]; /* default action allows yacc to see literal
characters */
%%

```